

Bài tập về Đa luồng

Bài 1: Tác vụ song song

Mô phỏng hai tác vụ chạy song song.

Yêu cầu:

1. Tạo lớp `Task` implements `Runnable` có 2 thuộc tính: `name` (String) và `durationMs` (long).
2. Trong `run()`: in `Start <name>`, gọi `sleep(durationMs)`, rồi in `End <name>`.
3. Trong `main`:
 - Tạo 2 `Task` khác nhau, bọc trong `Thread`.
 - Gọi `start()` cho cả hai.
 - Dùng `join()` để đợi cả hai hoàn thành.
 - In `All tasks done.` sau khi đợi xong.

Bài 2: Tổng của mảng

Tính tổng mảng bằng thread pool.

Yêu cầu:

4. Nhập `n` và `n` số nguyên từ bàn phím.
5. Chia mảng thành `k` đoạn (ví dụ `k = 4`).
6. Mỗi đoạn tạo một `Callable<Integer>` trả về tổng của đoạn.
7. Dùng `ExecutorService` (fixed thread pool) để `submit()` các `Callable`.
8. Dùng `Future.get()` để lấy kết quả và cộng lại thành tổng cuối.
9. In tổng cuối và đóng `ExecutorService` đúng cách.

Bài 3: Đồng bộ hóa

Mô phỏng tài khoản ngân hàng an toàn luồng.

Yêu cầu:

1. Tạo lớp `BankAccount` có thuộc tính số nguyên `balance`.
2. Cài các phương thức `deposit(int amount)` và `withdraw(int amount)` dạng `synchronized` để cập nhật `balance`.
3. Tạo 2 luồng:
 - Luồng A: lặp 1000 lần `deposit(100)`.
 - Luồng B: lặp 1000 lần `withdraw(100)`.
4. Dùng `join()` để đợi xong và in `final balance`.
5. Kỳ vọng: kết quả cuối đúng với logic (không bị sai do race condition).

Bài 4: Adapter – Chuyển đổi hệ thống in

Xây dựng cơ chế quản lý sách cho phép nhiều luồng đọc đồng thời và chặn khi có ghi dữ liệu.

Yêu cầu:

1. Tạo lớp `BookStore` quản lý `Map<String, Integer> stock` (key: tên sách, value: số lượng).
2. Dùng `ReentrantReadWriteLock`:
 - `readLock` cho `getStock(String title)`.
 - `writeLock` cho `addBook(String title, int qty)` và `borrow(String title, int qty)`.
3. Trong `main`:
 - Khởi tạo một vài sách có sẵn.
 - Tạo 3 luồng đọc (in số lượng sách) và 2 luồng ghi (mượn/nhập sách).
 - Chạy các luồng đồng thời và in log để thấy thứ tự thực thi.

Bài 5: Hệ thống xử lý đơn hàng

Vận dụng kiến thức: `ExecutorService`, `Callable/Future`, đồng bộ hóa, `AtomicInteger`.

Yêu cầu:

1. Nhập `m` đơn hàng, mỗi đơn có: `id` (String), `processMs` (long).
2. Dùng `ExecutorService` (fixed thread pool) để xử lý đơn hàng song song.
3. Mỗi đơn hàng là một `Callable<Boolean>`:
 - In `Start <id>`.
 - `sleep(processMs)`.
 - Nếu `processMs > 1500ms` thì coi là thất bại và trả về `false`, ngược lại trả về `true`.
4. Lưu kết quả vào một danh sách chung dạng `List<String> logs` theo mẫu:
 - `DONE <id>` hoặc `FAIL <id>`
 - Việc ghi vào `logs` phải đồng bộ (dùng `synchronized` hoặc `Lock`).
5. Dùng `AtomicInteger` để đếm số đơn thành công.
6. Sau khi `Future.get()` xong hết, in:
 - `Success = <count>`
 - Danh sách log theo thứ tự hoàn thành.
7. Đóng `ExecutorService` đúng cách.

Bài 6: Tổng số lớn thứ hai

Cho `n` mảng số nguyên. Với mỗi mảng, dùng một `Callable<Integer>` riêng để tìm **số lớn thứ hai (second largest)**. Sau khi tất cả hoàn thành, dùng `Future.get()` để lấy kết quả rồi cộng tất cả lại thành một tổng duy nhất.

Yêu cầu:

1. Nhập `n`, rồi nhập từng mảng (số phần tử + các phần tử).
2. Mỗi mảng được xử lý bởi một `Callable<Integer>` riêng biệt. Sử dụng `ExecutorService` để quản lý việc thực thi.

3. Dùng `Future.get()` để lấy kết quả và cộng tổng lại.
4. Trường hợp mảng không có số lớn thứ hai hợp lệ: xử lý sao cho chương trình không bị crash, bỏ qua mảng đó và tiếp tục.
5. In kết quả từng mảng và tổng cuối cùng.

Ví dụ

Input:

```
4
5 3 7 1 9 4
4 5 5 5 5
3 2 8 6
1 10
```

(Dòng đầu tiên nhập vào một số n là số mảng. Các dòng tiếp theo mỗi dòng nhập một số nguyên m là độ dài mảng và m số nguyên của mảng đó).

Output:

```
Array 0: second largest = 7
Array 1: Not found
Array 2: second largest = 6
Array 3: Not found
Sum = 13
```

Bài 7: Đếm số nguyên tố trong n mảng

Cho n mảng số nguyên dương. Đếm số lượng số nguyên tố trong từng mảng đồng thời, sau đó tìm mảng có **nhiều số nguyên tố nhất**. Nếu có nhiều mảng cùng đứng đầu thì in tất cả.

Yêu cầu: Việc đếm n mảng phải được thực hiện đồng thời — tức là n mảng phải được xử lý trong các luồng riêng biệt chạy song song, không phải lần lượt từng mảng một. Chương trình phải tổng hợp đủ kết quả của tất cả các luồng trước khi in ra kết luận cuối cùng.

Ví dụ

Input:

```
4
5 2 3 4 5 6
4 7 8 9 10
3 11 12 13
4 2 4 6 8
```

(Dòng đầu tiên nhập vào một số n là số mảng. Các dòng tiếp theo mỗi dòng nhập một số nguyên m là độ dài mảng và m số nguyên của mảng đó).

Output:

```
Array 0: 3
Array 1: 1
Array 2: 2
Array 3:
Most primes: Array 0 with 3 primes
```

Bài 8: Xử lý hai giai đoạn

Cho n mảng số nguyên. Thực hiện xử lý theo hai giai đoạn:

- **Giai đoạn 1:** Mỗi mảng được xử lý đồng thời để lọc ra các số nguyên tố.
- **Giai đoạn 2:** Nếu mảng có số lượng số nguyên tố **chẵn** thì tính tổng bình phương, nếu **lẻ** thì tính tổng lập phương. Sau khi tất cả hoàn thành, in ra tổng tất cả các giá trị tìm được.

Yêu cầu:

- Giai đoạn 1 và Giai đoạn 2 phải dùng **hai thread pool riêng biệt**.
- Việc thực hiện các giai đoạn cho từng mảng phải được thực hiện song song với nhau. Mảng nào xong Giai đoạn 1 thì in ra kết quả của Giai đoạn 1 luôn; xong Giai đoạn 2 thì in ra kết quả của Giai đoạn 2 luôn.

- (Optional): Pool của Giai đoạn 2 không được khởi tạo hoặc gọi submit bên trong task của Giai đoạn 1.

Ví dụ

Input:

```
3
5 2 3 4 5 6
4 7 8 9 10
3 11 12 13
```

(Dòng đầu tiên nhập vào một số n là số mảng. Các dòng tiếp theo mỗi dòng nhập một số nguyên m là độ dài mảng và m số nguyên của mảng đó).

Output:

```
Stage 1 - Array 0: [2, 3, 5]
Stage 1 - Array 1: [7]
Stage 2 - Array 1: sum of cubes = 343
Stage 1 - Array 2: [11, 13]
Stage 2 - Array 0: sum of cubes = 160
Stage 2 - Array 2: sum of squares = 290
Total = 793
```

(Thứ tự kết quả có thể thay đổi tùy thuộc vào máy, task nào xong trước thì in ra task đó luôn).

Bài 9: Counter an toàn với ReentrantLock

Mô phỏng bộ đếm được cập nhật bởi nhiều luồng.

Yêu cầu:

1. Tạo lớp Counter có biến int value
2. Sử dụng ReentrantLock để bảo vệ phương thức increment()
3. Tạo 4 luồng, mỗi luồng tăng counter 10000 lần

4. Dùng join() để đợi tất cả hoàn thành
5. In giá trị cuối cùng của counter
6. Thử dùng tryLock() để tránh chờ vô hạn (in thông báo nếu không lấy được lock)

Bài 10: Dừng luồng bằng biến volatile

Mô phỏng một luồng chạy liên tục cho đến khi được yêu cầu dừng

Yêu cầu:

1. Tạo lớp Worker implements Runnable gồm:
 - Biến boolean running = true;
 - Phương thức stop() để đặt running = false;
 - Trong run(): lặp while(running) và in "Working..."
2. Trong main:
 - Tạo Worker và chạy bằng Thread.
 - Cho luồng chạy khoảng 1 giây.
 - Gọi stop() để dừng luồng.
 - Đợi luồng kết thúc bằng join().
3. Sửa chương trình để biến running dùng từ khóa volatile.
4. Giải thích ngắn (comment trong code): Vì sao cần volatile?